CocoConnect

Technical Feasibility Document

Version 2 – Implementation Feasibility after Module Analysis

Client: Coconut Growers Association of Sri Lanka · Group 38

Prepared by: S.N. Vidanage (2023/IS/108)

Contents

Contents 2
Document History 3
1. Purpose 3
2. Summary of Findings 3
3. Changes Since Version 1 3
4. Product Marketplace – Implementation Feasibility 3
4.1 Why negotiation needs extra care 4
5. Service Marketplace – Implementation Feasibility 4
6. Component Reuse 4
7. Dependencies and Build Order 5
8. Data and Integration Feasibility 5
9. Updated Technical Risks 5
10. Conclusion 5

Document History

Version	Content
1	Overall feasibility: technology stack, modules, external integrations, cost, security, performance, team skills and risks
2	This document: feasibility of each marketplace function at implementation level, after the module analysis and technical review – components, effort, dependencies, build order, decisions changed since version 1 and updated risks

1. Purpose

Version 1 showed that CocoConnect can be built with the chosen stack. After the module analysis (Functional Analysis Document v2) and the technical review with stakeholders, this version checks feasibility at a finer level: can each function of the Product Marketplace and Service Marketplace be implemented with the planned components, how much effort it needs, and in what order it should be built.

2. Summary of Findings

- **All Product Marketplace functions are feasible.** They use standard database operations, REST endpoints and screens the team can build.
- **All Service Marketplace functions are feasible,** mostly by reusing the Product Marketplace's patterns (distance filter, request and accept flow, notifications).
- **Two functions need extra care:** price negotiation (offers must stay consistent for both sides) and the service calendar (no double bookings).
- **Online payment remains deferred.** Payment stays outside the app in this release.

3. Changes Since Version 1

Topic	Version 1	Version 2	Change
Maps	OpenStreetMap recommended over Google Maps	Confirmed: OpenStreetMap with flutter_map; Nominatim for place names	**Unchanged**
Location capture	Not detailed	Location picked on a map at sign-up; GPS as an option	**New**
Price agreement	Contact sellers	In-app negotiation chat with price offers and Accept / Reject	**Changed**
Request matching	Search and filter	Automatic matching by quantity and a 7-day date window	**New**
Deal completion	Not detailed	Estate owner records delivered quantity and final price; admin reviews price changes	**New**
Service marketplace	Profiles and job requests	Adds a map of nearby machinery with rates and a service calendar	**Changed**
Online payment	Low feasibility	Still deferred	**Unchanged**
Offline harvest recording	Medium feasibility	Still medium; built after the marketplaces	**Unchanged**

4. Product Marketplace – Implementation Feasibility

Function	Frontend	Backend / database	Feasibility	Effort
Post sell offer	Sell Summary form	Insert into product_listings	**High**	**Small**
Browse by coconut type	Coconut Categories, listing list	Filter product_listings by type	**High**	**Small**
Filter by distance, price, date	Distance chips, sort menu	Haversine distance on the server; filter and sort before returning	**High**	**Medium**
Post buy request	Request form with date picker and validation	Insert into buyer_requests	**High**	**Small**
Match requests and listings	Interested Buyers list; nearby listings	Quantity at least the request; date within 7 days	**High**	**Medium**

Function	Frontend	Backend / database	Feasibility	Effort
Send offer on a request	Buyer request card with Send Offer	Insert into sell_requests; open a chat	**High**	**Medium**
Negotiation chat with offers	Chat screen, offer bubbles, Accept / Reject	negotiations and negotiation_messages with an offer status	**Medium**	**Large**
Accept offer → create order	Accept Deal button	One database transaction: create order, update offers and request	**Medium**	**Medium**
Orders hub	Pending Orders, Deals, Transactions tabs; due-date banner	Read orders for either side with due dates	**High**	**Medium**
Complete deal	Complete Deal form	Update order; flag price dispute if the price differs	**High**	**Medium**
Market price strip	Live deal strip	Latest order and average price per nut; refresh every few seconds	**High**	**Small**
Alerts	Alerts tab and badges	Read pending offers and unread messages	**High**	**Medium**

4.1 Why negotiation needs extra care

- Both people can send offers, so the system must know which offer is current. A new offer marks the older pending one as replaced.
- Only the person receiving an offer may accept it, otherwise someone could accept their own price.
- Accepting must create the order and close competing offers together. A database transaction guarantees this: either every step is saved or none is.
- The app has no push service yet, so screens refresh regularly and when reopened to show new messages.

5. Service Marketplace – Implementation Feasibility

Function	Frontend	Backend / database	Feasibility	Effort
Service profile	Profile form (skill, charging method, rate)	Service_Profile linked to the worker	**High**	**Small**
List machinery	Equipment form	Equipment linked to the provider	**High**	**Small**
Search providers by category	Category chips, provider cards	Filter by skill type	**High**	**Small**
Nearby machinery on a map	OpenStreetMap with machine markers and price tags	Same distance calculation as the marketplace	**High**	**Medium**
Hiring request	Request form with estimated cost	Hiring_Request with status	**High**	**Small**
Accept / reject request	Provider's request list	Update status; create Job_Assignment	**High**	**Small**
Service calendar	Month view with confirmed and pending marks	Job_Assignment dates; block overlapping bookings	**Medium**	**Medium**
Notifications	Alerts tab	Reuse the marketplace's alert logic	**High**	**Small**
Ratings after a job	Rating prompt	Rating linked to Job_Assignment	**High**	**Small**

6. Component Reuse

Building the Product Marketplace first gives components the Service Marketplace can reuse, which lowers its effort and risk.

Reusable part	Built for	Reused in
Login, roles and access checks	All modules	Every service marketplace route
Distance calculation and radius filter	Nearby listings	Nearby workers and machinery
Map with markers (flutter_map)	Location picker at sign-up	Machinery map

Reusable part	Built for	Reused in
Request → accept → status flow	Buyer requests and offers	Hiring requests
Alerts and badges	Offers and messages	Hiring request alerts
Card, chip, form and bottom-bar widgets	Marketplace screens	All service screens
Orders hub with due dates	Deals	Service calendar and job list

7. Dependencies and Build Order

1. User management, roles and location at sign-up – everything depends on knowing who the user is and where they are.
2. Listings and buy requests, with matching and distance filters.
3. Negotiation chat, offers and order creation.
4. Orders hub, deal completion and price disputes.
5. Alerts and the market price strip.
6. Service profiles, machinery and the provider map, reusing steps 1–5.
7. Hiring requests, job status and the service calendar.
8. Ratings for completed orders and jobs.
9. Offline harvest recording.

8. Data and Integration Feasibility

- Existing team tables (product_listings, buyer_requests, orders) can be extended with new columns instead of being replaced, so team members' work is not broken.
- Database changes are written as numbered SQL scripts that are safe to run more than once, so every copy of the database reaches the same structure.
- Every API route is documented in Swagger first, so frontend and backend can be built in parallel against an agreed contract.
- An in-memory test database lets the backend be tested without touching the shared database.

9. Updated Technical Risks

Risk	Likelihood	Impact	Mitigation
Buyer and owner see different offer states	Medium	High	Single source of truth in the database; offer statuses; transactions on accept
New messages not seen without push notifications	Medium	Medium	Regular refresh and refresh on reopening; push notifications later
Double booking of a provider	Medium	Medium	Check for overlapping dates before accepting
Merging separately built modules	Medium	High	Shared API contract in Swagger; additive database changes; frequent merges
Location inaccurate or missing	Medium	Medium	Pick location on a map; show 'distance unknown' instead of failing
Scope growth in the service marketplace	Medium	High	Build the map and calendar with core data first; extra features later

10. Conclusion

The detailed review confirms that every Product Marketplace and Service Marketplace function is technically feasible with the planned components. Negotiation and the service calendar carry the most risk and are handled with database transactions and booking checks. Building the Product Marketplace first and reusing its parts makes the Service Marketplace faster and safer to deliver. Online payment stays deferred.